

Static view: Utilities

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<Module>>

<<uses>>

oradio_logging.py
<<module>>

+ oradio_log.debug()
+ oradio_log.warning()
+ oradio_log.info()
+ oradio_log.error()

system_sounds.py
<<module>>

+ play_sound()

Every modules/classes can use these utilities

Hardware Abstraction Layer

Static view: Throttle

Application Layer

oradio_control.py
«module»

+ execute(): void

Middle-ware Layer

«uses»

throttled_monitor.py
«module»

- throttled_monitor: RpiThrottlingMonitor()

«uses»

throttled_monitor.RpiThrottlingMonitor
«singleton»

- stop_event: Event()

- init()
+ start()
+ stop()
+ run()
+ force_throttled()
+ clear_throttled()

The rpi-throttling-monitor thread
check the throttle state in a loop

«consists of»

«thread»

rpi-throttling-monitor

+ stop_event: Event()
- vcgencmd
- get_throttle_value()

Hardware Abstraction Layer

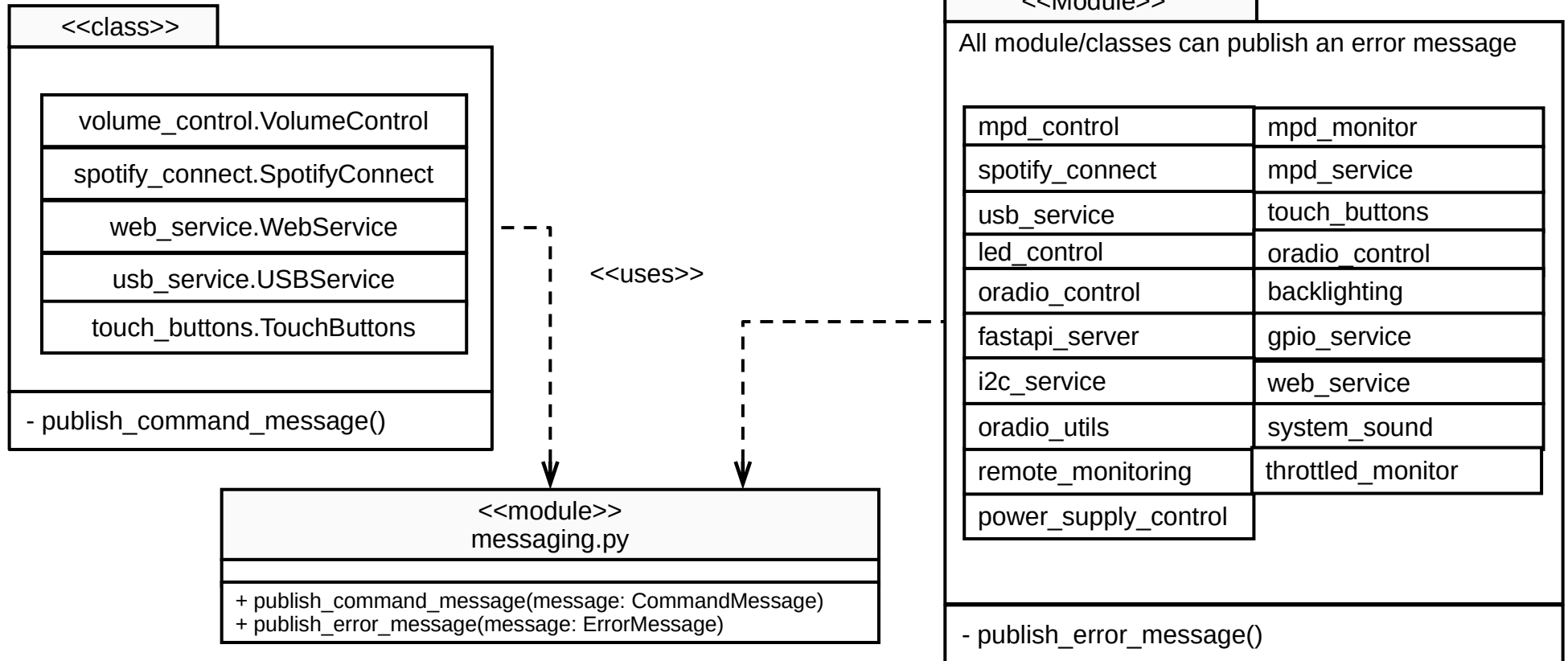
Static view: Messages

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



Hardware Abstraction Layer

Static view: Remote monitoring

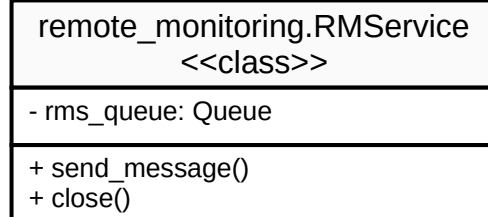
Application Layer

oradio_control.py
<<module>>

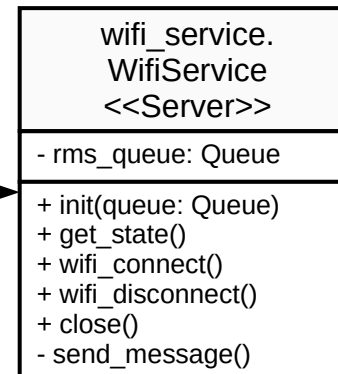
+ execute(): void

Middle-ware Layer

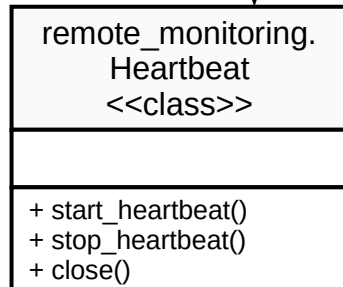
<<uses>>



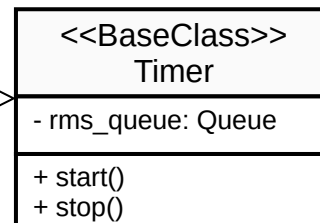
<<uses>>



<<uses>>



<<is a>>



Hardware Abstraction Layer

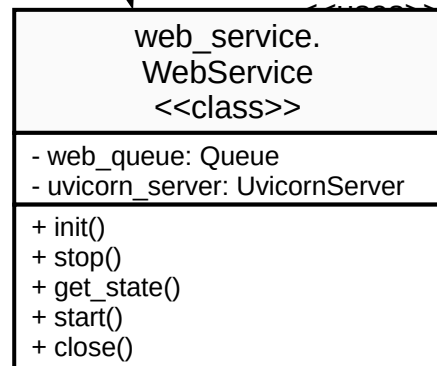
Static view: Web Service

Application Layer

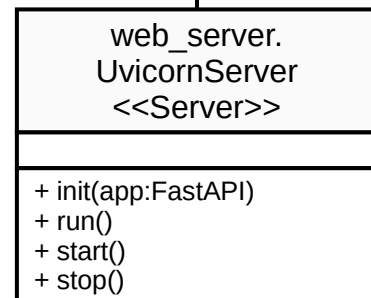
oradio_control.py
<<module>>

+ execute(): void

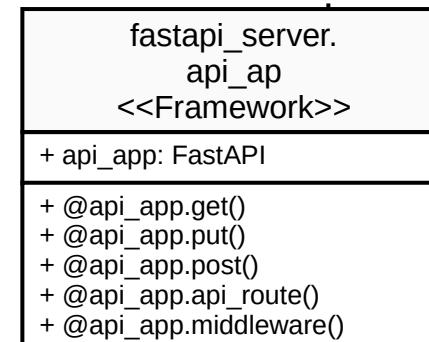
Middle-ware Layer



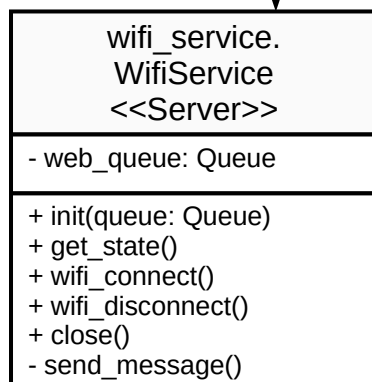
<<consists of>>



<<uses>>



<<uses>>



Hardware Abstraction Layer

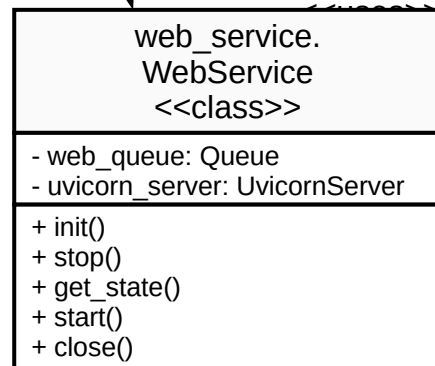
Static view: WiFi Service

Application Layer

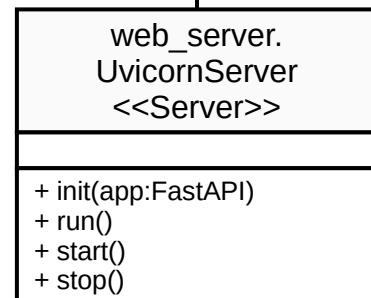
oradio_control.py
<<module>>

+ execute(): void

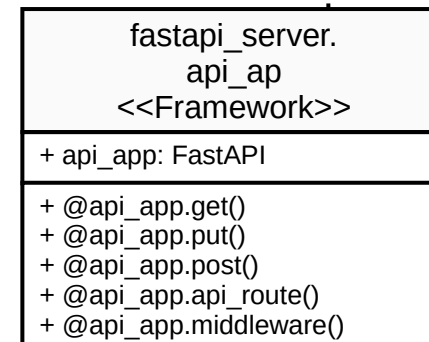
Middle-ware Layer



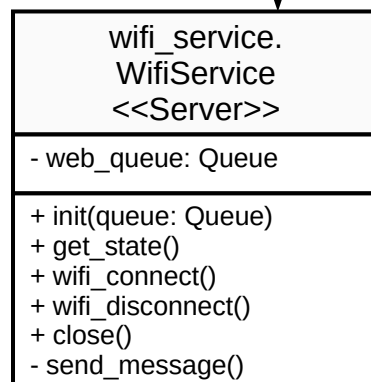
<<consists of>>



<<uses>>



<<uses>>



Hardware Abstraction Layer

Static view: USB

Application Layer

oradio_control.py
«module»

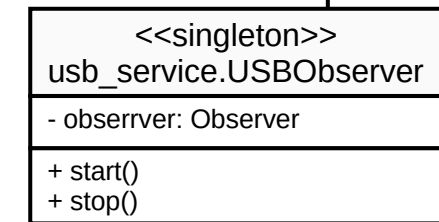
+ execute(): void

Middle-ware Layer

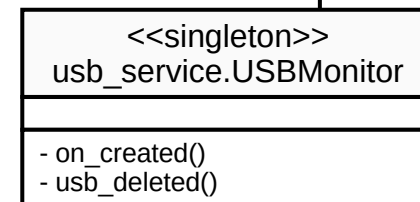
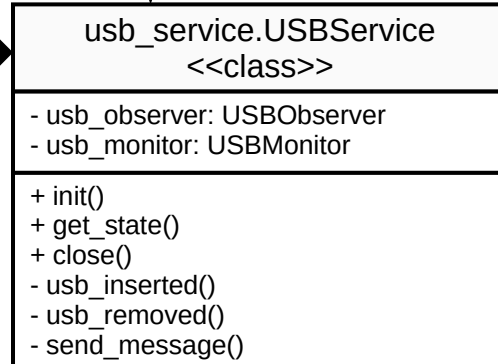
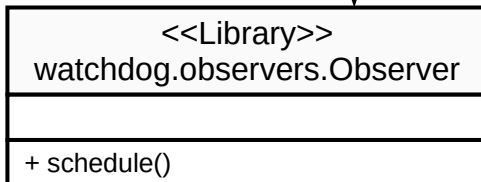
«uses»

«consists of»

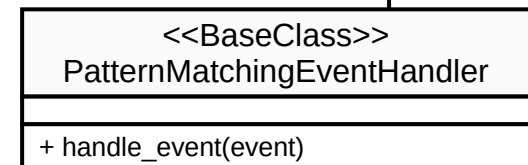
«consists of»



«uses»



«is a»



Hardware Abstraction Layer

Static view: Volume

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

Waarom start(), stop(), set_notify().
Staat altijd op notify. Dus notify default
aan. Gaat nooit gestopt worden

volume_control.VolumeControl
<<class>>

+ set_notify()
+ start()
+ stop()

<<has>>

i2c_service.I2CService
<<class>>

+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)

Hardware Abstraction Layer

Static view: Error

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

error_service.py.ErrorService
<<class>>

- error_queue:subscriber_errors()

- init()
- error_handler()

<<thread>>

error_service.error_handler

<<uses>>

<<module>>
messaging.py

+ ErrorMessage

+ safe_get(error_queue: Queue)

Hardware Abstraction Layer

Static view: Messaging

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

messaging
<<module>>

- CommandMessage()
- ErrorMessage()
+ subscribe_commands() -> Queue
+ subscribe_errors() -> Queue
+ unsubscribe_commands(queue: Queue)
+ unsubscribe_errors(queue: Queue)
+ publish_command(message: CommandMessage)
+ publish_error(message: ErrorMessage)
+ safe_get(queue: Queue)

<<has>>

messaging.PubSubManager
<<singleton>>

- Topic.COMMAND: []
- Topic.ERROR: []
+ subscribe(topic: Topic) -> Queue
+ unsubscribe(topic: Topic, queue: Queue)
+ publish(topic: Topic, message: CommandMessage | ErrorMessage)

<<uses>>

messaging.Topic
<<data>>

+ COMMAND = "COMMAND"
+ ERROR = "ERROR"

messaging.ErrorMessage
<<data>>

+ source: String
+ message: String
+ is_valid()

messaging.CommandMessage
<<data>>

+ source: String
+ message: String
+ data: Any
+ is_valid()

Hardware Abstraction Layer

Static view: Power Supply

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

power_supply_control.PowerSupplyService
<<class>>

+ set_nom_voltage()
+ set_max_voltage()
+ set_standby_voltage()
+ read_status()

<<has>>

<<singleton>>

i2c_service.I2CService

+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)

Hardware Abstraction Layer

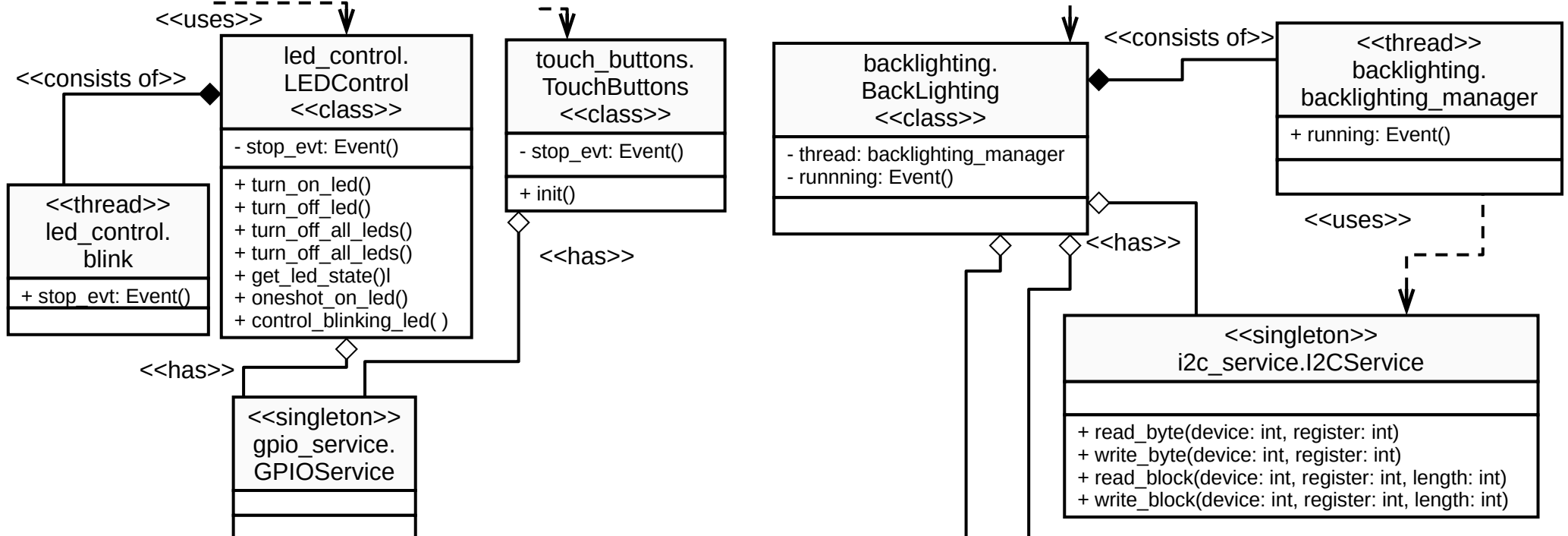
static view: Front-panel

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



Hardware Abstraction Layer

Waarom backlighting thread met stop en off. Staat eigenlijk altijd aan! Wordt ook in de module gestart!

Static View: Music Players

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

mpd_control.MPDControl
<<class>>

+ init(class: MPDService)
+ is_web_radio()
+ next()
+ play()
+ stop()
+ pause()

<<uses>>

mpd_monitor.MPDMonitor
<<class>>

- running: Event
+ init(class: MPDService)
- listen()

<<thread>>
mpd_monitor.
listen

+ running: Event

<<uses>>

spotify_connect.SpotifyConnect
<<class>>

+ play()
+ pause()
+ get_state()

<<is A>>

mpd_service.MPDService
<<class>>

- client: MPDClient
+ init()
+ get_stats()
- execute()

mpd.MPDClient
<<library>>

+ init()
+ idle()
+ update()
+ next()
+ play()
+ stop()
+ pause()
+ status()

<<consists of>>

<<is A>>

Hardware Abstraction Layer